Unix & Shell Programming

Module 7 – UNIX Processes

Question & Answer Notes

BCA Semester 6 · MAKAUT

Q1. What is a process in UNIX? Explain process types.

A **process** is a program that is currently running in memory. When you execute a command or program, UNIX creates a process for it. Every process has a unique number called **PID (Process ID)**.

Process Types:

1. **Foreground Process**
   - Runs directly in the terminal and takes user input.
   - User must wait for it to finish before running next command.
   - Example: `ls`, `cat file.txt`
2. **Background Process**
   - Runs in the background without blocking the terminal.
   - Started by adding `&` at the end of a command.
   - Example: `sleep 100 &`
3. **Daemon Process**
   - System background process that runs continuously.
   - Not attached to any terminal.
   - Examples: `httpd` (web server), `sshd` (SSH server), `crond` (cron scheduler).
4. **Zombie Process**
   - A process that has finished execution but its entry still exists in the process table because the parent has not read its exit status.
5. **Orphan Process**
   - A child process whose parent has terminated. It is adopted by the `init` process (PID 1).

> Every process in UNIX is created by another process. The first process is `init` (PID = 1), which is the parent of all processes.

Q2. Explain process listing commands with options.

The `ps` command (Process Status) is used to list running processes.

Basic Syntax: `ps [options]`

Command	Description
`ps`	Shows processes of current terminal
`ps -e`	Shows all processes on the system
`ps -f`	Full format listing (shows more columns)
`ps -ef`	All processes in full format
`ps -u user`	Shows processes of a specific user
`ps aux`	Detailed list of all processes (BSD style)
`ps -p PID`	Show info of a specific process by PID

Output columns explained (for `ps -ef`):

Column	Meaning
`UID`	User who owns the process
`PID`	Process ID
`PPID`	Parent Process ID
`C`	CPU utilization
`STIME`	Start time of process
`TTY`	Terminal associated with process
`TIME`	Total CPU time used
`CMD`	Command that started the process

Example:

```
ps -ef              # list all processes with full details
ps aux              # list all with CPU and memory usage
ps -u student       # list processes owned by user "student"
ps -p 1234          # info about process with PID 1234
```

Q3. What is the init process? Explain UNIX login process.

The init Process:

`init` is the **first process** started by the UNIX kernel when the system boots. It always has **PID = 1**. All other processes are descendants of `init`. It is responsible for starting system services and maintaining the system run level.

Modern Linux systems use `systemd` instead of traditional `init`, but the concept is the same.

UNIX Login Process (Step by Step):

1. **System Boot** – Kernel loads into memory and starts running.
2. **init starts** – Kernel launches `init` (PID 1).
3. **getty runs** – `init` spawns `getty` process for each terminal. `getty` opens the terminal and displays the `login:` prompt.
4. **User enters username** – `getty` reads the username and passes it to `login` program.
5. **login program runs** – It asks for the **password** and checks it against `/etc/passwd` and `/etc/shadow`.

6. **Authentication success** – If credentials are correct, `login` sets up the environment (home directory, shell, environment variables).
7. **Shell starts** – `login` executes the user's shell (e.g., `/bin/bash`) as specified in `/etc/passwd`.
8. **User gets prompt** – The shell displays the command prompt and waits for input.

If login fails, `getty` displays an error and shows the login prompt again. The whole cycle restarts.

Q4. Explain fork(), getpid(), and getppid().

These are important **system calls** used in C programs to create and manage processes in UNIX.

1. fork()

- Creates a new child process by duplicating the calling (parent) process.
- After `fork()`, both parent and child run the same code from the next line.
- Returns **0** to the child process and **child's PID** to the parent. Returns **-1** on failure.

2. getpid()

- Returns the **PID** (Process ID) of the calling process.

3. getppid()

- Returns the **PPID** (Parent Process ID) of the calling process.

Example C Program:

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    int pid;
    pid = fork();   // create child process

    if (pid == 0) {
        // This block runs in child
        printf("Child Process\n");
        printf("Child PID  = %d\n", getpid());
        printf("Child PPID = %d\n", getppid());
    } else {
        // This block runs in parent
        printf("Parent Process\n");
        printf("Parent PID       = %d\n", getpid());
        printf("Child's PID seen = %d\n", pid);
    }
    return 0;
}
```

Sample Output:

```
Parent Process
Parent PID       = 1200
Child's PID seen = 1201
Child Process
Child PID  = 1201
Child PPID = 1200
```

Q5. What is the wait() system call? Explain zombie processes.

wait() System Call:

`wait()` is a system call used by a **parent process** to pause its execution until one of its **child processes** finishes. It also collects the exit status of the child.

- Header file: `<sys/wait.h>`
- Syntax: `pid_t wait(int *status);`
- Returns the PID of the child that terminated.

Example:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pid = fork();
    if (pid == 0) {
        printf("Child running...\n");
        sleep(2);
        printf("Child done.\n");
    } else {
        printf("Parent waiting...\n");
        wait(NULL);   // parent waits for child
        printf("Parent: child has finished.\n");
    }
    return 0;
}
```

Zombie Process:

A **zombie process** is a child process that has **completed execution** but whose entry still remains in the **process table** because the parent has not yet called `wait()` to read its exit status.

- It appears as `Z` or `<defunct>` in `ps` output.
- It consumes no CPU or memory, but occupies a slot in the process table.
- Too many zombies can exhaust the process table.
- **Solution:** The parent should call `wait()` after forking, or the `init` process eventually cleans them up.

A zombie is different from an orphan. An orphan's parent is dead; a zombie's parent is alive but has not called `wait()` yet.

Q6. Explain pipe() and message passing.

pipe() – Inter-Process Communication (IPC):

A **pipe** is a method of communication between two processes where the output of one process becomes the input of another. Data flows in one direction only (unidirectional).

System call syntax:

```
int pipe(int fd[2]);
// fd[0] = read end
// fd[1] = write end
```

Example C Program:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    char msg[] = "Hello from parent!";
    char buf[50];

    pipe(fd);   // create the pipe

    if (fork() == 0) {
        // Child reads from pipe
        close(fd[1]);           // close write end
        read(fd[0], buf, sizeof(buf));
        printf("Child got: %s\n", buf);
    } else {
        // Parent writes to pipe
        close(fd[0]);           // close read end
        write(fd[1], msg, strlen(msg) + 1);
    }
    return 0;
}
```

Shell pipe (—):

```
ls -l | grep ".txt"     # output of ls goes as input to grep
ps aux | grep "bash"    # find bash processes
```

Message Passing:

Message passing is another form of IPC where processes communicate by **sending and receiving messages** through a message queue managed by the OS kernel.

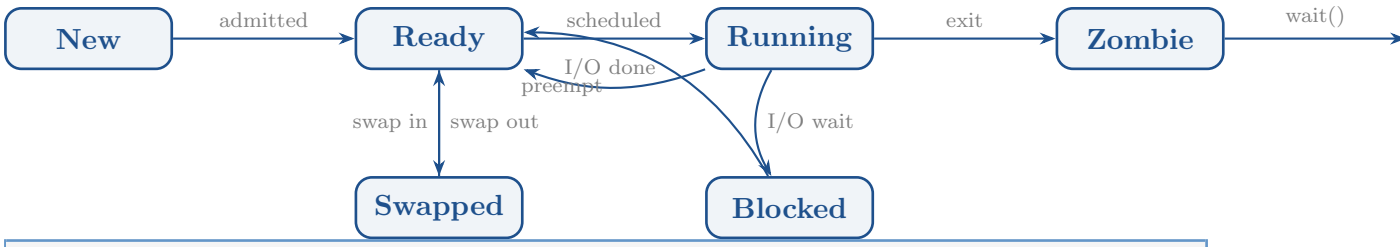
- Uses system calls: `msgget()`, `msgsnd()`, `msgrcv()`, `msgctl()`
- Messages are stored in a queue in the kernel.
- Both related and unrelated processes can communicate.
- More flexible than pipes – supports multiple senders and receivers.

Q7. Explain UNIX process states with a diagram.

A process goes through different states during its lifetime in UNIX:

State	Description
New / Created	Process is being created (fork called)
Ready	Process is in memory, waiting for CPU
Running	Process is currently executing on CPU
Blocked/Sleep	Waiting for I/O or an event to complete
Swapped Out	Moved to disk to free up memory
Zombie	Finished but parent not yet called wait()
Terminated	Process has ended completely

State Transition Diagram:



In `ps` output, process state codes are: `R` = Running, `S` = Sleeping, `D` = Uninterruptible sleep, `Z` = Zombie, `T` = Stopped.

Q8. Discuss the UNIX scheduler.

The **UNIX scheduler** is the part of the kernel that decides which process gets to use the CPU and for how long. It is also called the **process scheduler**.

Goal of the Scheduler:

- Maximize CPU utilization.
- Give fair CPU time to all processes.
- Ensure interactive processes respond quickly.
- Handle real-time process requirements.

Scheduling Concepts:

1. **Time Slice / Quantum** – Each process gets a fixed amount of CPU time called a time slice. When it expires, the next process runs.
2. **Priority** – Every process has a priority. Higher priority processes get CPU time first. Priority in UNIX ranges from **-20** (highest) to **+19** (lowest).
3. **Preemptive Scheduling** – If a higher priority process becomes ready, the current process is preempted (stopped temporarily) and the high-priority process runs.

4. **Multi-level Queue** – Processes are grouped into different queues based on their priority. The scheduler picks from the highest priority non-empty queue.

Types of Schedulers:

Scheduler	Role
Long-term (Job) scheduler	Decides which jobs enter the ready queue
Short-term (CPU) scheduler	Decides which ready process runs on CPU next
Medium-term scheduler	Decides which process to swap in/out of memory

The modern Linux kernel uses the **CFS (Completely Fair Scheduler)**, which gives each process a fair share of CPU time proportional to its weight (priority).

Q9. What is swapped memory?

Swapped Memory (also called **swap space**) is a portion of the **hard disk** used as an extension of physical RAM (main memory).

Why it is needed:

- When RAM is full and a new process needs memory, the OS moves some inactive processes or pages from RAM to the swap space on disk. This is called **swapping out**.
- When that process is needed again, it is moved back from disk to RAM. This is called **swapping in**.
- This way, the system can run more processes than RAM alone would allow.

Key Points:

- Swap is much **slower** than RAM because disk access is slow.
- Heavy swapping (called **thrashing**) makes the system very slow.
- In Linux, swap can be a **swap partition** or a **swap file**.
- Typical swap size is 1x or 2x the physical RAM.

Commands to check swap:

```
free -h          # shows RAM and swap usage in human-readable
   format
swapon --show    # lists active swap spaces
cat /proc/swaps # another way to see swap info
```

Sample output of `free -h`:

```
               total    used    free    shared  buff/cache
                  available
Mem:            3.8G    1.2G    1.5G     150M        1.0G       2.3G
Swap:           2.0G    200M    1.8G
```

If your system constantly uses a lot of swap, it means you need more RAM. Swap is a safety net, not a replacement for RAM.

Q10. Explain the vmstat and top commands.

1. vmstat (Virtual Memory Statistics)

`vmstat` reports information about processes, memory, paging, I/O, and CPU activity.

Syntax: `vmstat [delay] [count]`

```
vmstat            # single snapshot
vmstat 2 5        # report every 2 seconds, 5 times
```

vmstat output columns:

Column	Meaning
`r`	Number of processes waiting for CPU (run queue)
`b`	Number of processes in uninterruptible sleep
`swpd`	Amount of virtual (swap) memory used (KB)
`free`	Free (idle) memory (KB)
`si`	Memory swapped in from disk (KB/s)
`so`	Memory swapped out to disk (KB/s)
`us`	CPU time spent in user space (%)
`sy`	CPU time spent in kernel/system space (%)
`id`	CPU idle time (%)
`wa`	CPU time waiting for I/O (%)

2. top Command

`top` gives a **real-time, continuously updating** display of system resource usage and running processes. It is like the Task Manager in Windows.

Syntax: `top`

```
top               # start top (press q to quit)
top -u student    # show only student's processes
top -p 1234       # monitor a specific PID
```

top display shows:

- System uptime and number of users logged in.
- Total tasks: running, sleeping, stopped, zombie.
- CPU usage breakdown: user, system, idle, wait.
- Memory and swap usage.
- Per-process list sorted by CPU usage by default.

Useful keys inside top:

Key	Action
`q`	Quit
`k`	Kill a process (enter PID)
`M`	Sort by memory usage
`P`	Sort by CPU usage
`u`	Filter by user
`h`	Help

Q11. Explain the nice command with examples.

nice is a command used to **set the priority** of a process when it is started. The priority value is called the **nice value** or **niceness**.

Nice value range: -20 (highest priority) to **+19** (lowest priority).

- Default nice value for a new process is **0**.
- A **lower (more negative)** nice value means higher priority (more CPU time).
- A **higher (more positive)** nice value means lower priority (less CPU time).
- Only the **root (superuser)** can set negative nice values.
- Regular users can only increase the nice value (lower priority).

Syntax:

```
nice -n value command
```

Examples:

```
# Run a script with lower priority (nice = 10)
nice -n 10 ./backup.sh

# Run a program with higher priority (root only)
sudo nice -n -5 ./important_task

# Run long compilation with very low priority
nice -n 19 make

# Check nice value of a running process
ps -o pid,ni,cmd -p 1234
```

renice – Change priority of a running process:

`renice` is used to change the nice value of an **already running** process.

```
renice 5 -p 1234        # set nice value of PID 1234 to 5
renice -10 -p 5678      # increase priority (root only)
renice 10 -u student    # lower priority of all student's
   processes
```

Nice Value	Priority	Who can set it
-20 to -1	High priority	Root only
0	Default	Any user
1 to 19	Low priority	Any user

Remember: `nice` is used when *starting* a process; `renice` is used to change priority of an *already running* process.

End of Module 7 Notes